

TALLER 3

Clustering, Optimización Combinatoria y Algoritmos Evolutivos
Maestría en Ciencia de Datos – USFQ | Mayo 2026

Integrante	Problema
Anderson Alvarez	P1 – Clustering No Supervisado y Detección de Anomalías (K-Means / Jerárquico)
Maximo Pinta	P2 – Optimización TSP: Modelo Exacto MIP/MTZ + Heurística NN + 2-opt
Steeven Quezada	P3 – Algoritmo Genético para Optimización de Cadenas de Texto

Este documento presenta las soluciones desarrolladas para el Taller 3 de la Maestría en Ciencia de Datos de la USFQ. Se abordan tres problemas: aprendizaje no supervisado mediante clustering e identificación de anomalías, optimización combinatoria del TSP con modelo exacto e híbrido heurístico, y algoritmos genéticos con operadores mejorados para convergencia rápida. En cada caso se detalla el pipeline implementado, las decisiones algorítmicas justificadas y las alternativas evaluadas.

PROBLEMA 1: Clustering No Supervisado y Detección de Anomalías

1. Enunciado del Problema

Se construyó un pipeline de aprendizaje no supervisado sobre datos operativos (CO2, temperatura, ocupación, entre otras variables) para identificar patrones de comportamiento diario e intervalos anómalos. El objetivo fue segmentar los registros en grupos interpretables para operación y detectar automáticamente lecturas fuera de rango sin etiquetas previas.

2. Complejidad

Algoritmo	Complejidad por iter.	Convergencia	Sensibilidad a outliers <i>Alta — requiere limpieza previa</i>
K-Means (k=3)	$O(n \cdot k \cdot d)$	Rápida (< 20 iter. típico)	previa
Agglomerative Clustering	$O(n^2 \log n)$	No iterativo (dendrograma)	Moderada
Detección por centroide	$O(n \cdot k)$	Instantánea post-clustering	Controlada por percentil

Donde n = número de registros, k = número de clusters, d = número de dimensiones (variables). Con datos estandarizados, K-Means converge rápidamente y produce fronteras interpretables.

3. Pipeline Implementado

El pipeline siguió cuatro etapas secuenciales:

- **Carga y exploración:** lectura del dataset operativo, revisión de tipos, valores nulos y distribución de variables.
- **Limpieza de outliers:** eliminación de registros extremos antes de clusterizar para evitar deformación de centroides.
- **Estandarización (StandardScaler):** normalización a media=0, desv.=1 para que CO2 (escala grande) no domine sobre temperatura (escala pequeña).
- **Clustering:** K-Means con $k=3$ para análisis multivariable operativo; Clustering Jerárquico para validación visual mediante dendrograma.
- **Detección de anomalías:** cálculo de distancia de cada punto a su centroide asignado; umbral = percentil 95 de distancias.

4. Alternativas de Algoritmos

Algoritmo	Ventaja <i>Fronteras limpias, rápido, interpretable</i>	Por qué no se priorizó
K-Means $k=3$ (*)	para operación diaria	Elección principal — óptimo para este dataset

Algoritmo	Ventaja	Por que no se priorizo
K-Means k=7 (dias)	Alineado con la semana calendario	Menos robusto, el patron real depende mas de horario/ocupacion que del dia
DBSCAN / HDBSCAN	Detecta clusters de forma arbitraria; robusto a outliers	Requiere calibrar eps/min_samples; menos transparente para reporte academico
Isolation Forest	Mas potente para anomalias en alta dimension	Menos explicable que distancia a centroide para justificacion inicial
One-Class SVM	Robusto para distribucion no normal	Mayor costo computacional; hiperparametros criticos

5. Eleccion y Justificacion

K-Means con k=3 fue la eleccion principal por tres razones:

- **Estandarizacion critica:** sin StandardScaler, CO2 domina la geometria del espacio y los clusters no reflejan el comportamiento real.
- **k=3 vs k=7:** tres clusters (baja ocupacion / normal / alta ocupacion) son mas interpretables operativamente que siete dias fijos, ya que el patron real depende mas de horario y carga que del dia de la semana.
- **Deteccion por centroide (percentil 95):** simple, trazable y justificable: cualquier punto cuya distancia supere el percentil 95 de su cluster se marca como anomalia.

6. Implementacion

Estandarizacion y K-Means:

```
from sklearn.preprocessing import StandardScaler from sklearn.cluster import KMeans scaler = StandardScaler() X_scaled = scaler.fit_transform(X_clean) kmeans = KMeans(n_clusters=3, random_state=42, n_init=10) labels = kmeans.fit_predict(X_scaled) centers = kmeans.cluster_centers_
```

Deteccion de anomalias por distancia a centroide:

```
import numpy as np distances = np.linalg.norm(X_scaled - centers[labels], axis=1) threshold = np.percentile(distances, 95) anomalies = distances > threshold print(f'Anomalias detectadas: {anomalies.sum()}')
```

Clustering Jerarquico (validacion):

```
from sklearn.cluster import AgglomerativeClustering from scipy.cluster.hierarchy import dendrogram, linkage Z = linkage(X_scaled, method='ward') hier = AgglomerativeClustering(n_clusters=3, linkage='ward') labels_h = hier.fit_predict(X_scaled)
```

7. Resultados Obtenidos

Configuración	k	Método anomalías	Interpretabilidad	Observación
Hipótesis inicial	7	—	Baja	Segmentación por días; no captura patrón horario
Análisis operativo	3	Dist. centroide p95	Alta	Tres perfiles claros: baja / normal / alta ocupación

8. Conclusiones del Problema 1

El pipeline de limpieza + estandarización + K-Means ($k=3$) produjo clusters operativamente interpretables. La decisión de estandarizar fue crítica: sin este paso, las diferencias de escala entre variables distorsionan la geometría del espacio y los centroides no representan correctamente los perfiles de comportamiento. La detección de anomalías por percentil 95 ofrece trazabilidad completa, a diferencia de métodos de caja negra como Isolation Forest.

PROBLEMA 2: Optimización TSP — Modelo Exacto MIP/MTZ + NN + 2-opt

1. Enunciado del Problema

Se implementa una solución completa para el Problema del Viajante (TSP) combinando un modelo de programación entera (MIP) exacto formulado con restricciones MTZ (Miller-Tucker-Zemlin) en Pyomo + GLPK, y un enfoque híbrido de heurística Nearest Neighbor con mejora local 2-opt. Además se incorpora una variante de cortes anti-cruces para eliminar aristas que se intersectan.

2. Complejidad

Enfoque	Complejidad	Optimalidad	Uso recomendado
MIP/MTZ exacto (Pyomo+GLPK)	Exponencial en el peor caso	Óptimo global	Instancias pequeñas ($n < 30$) o como referencia de calidad
Nearest Neighbor (NN)	$O(n^2)$	Sub-óptimo	Solución inicial rápida para cualquier n
2-opt (first improvement)	$O(n^2)$ por iteración	Óptimo local	Post-proceso sobre NN; elimina cruces de aristas
Cortes anti-cruces	$O(n^2)$	Óptimo local mejorado	Refinamiento adicional sobre 2-opt

3. Modelo Exacto MTZ (Pyomo)

El modelo MIP con formulación MTZ elimina subtours mediante variables auxiliares de orden $u(i)$. Variables binarias $x(i,j)$ indican si el arco $i \rightarrow j$ pertenece a la ruta óptima:

```
import pyomo.environ as pyo
model = pyo.ConcreteModel()
model.x = pyo.Var(N, N, domain=pyo.Binary) # arcos
model.u = pyo.Var(N, domain=pyo.NonNegativeReals) # orden MTZ
# Objetivo: minimizar distancia total
model.obj = pyo.Objective(
    expr=sum(dist[i][j]*model.x[i,j] for i in N for j in N if i!=j),
    sense=pyo.minimize
) # Restricción MTZ anti-subtour
def mtz_rule(m, i, j):
    if i != j and i != 0 and j != 0:
        return m.u[i] - m.u[j] + n*m.x[i,j] <= n - 1
    return pyo.Constraint.Skip
model.mtz = pyo.Constraint(N, rule=mtz_rule)
```

Rutas obtenidas con el modelo LP exacto (GLPK) para instancias pequeñas: a $n=10$ y $n=20$ el solver encuentra la solución óptima dentro del límite de tiempo. A partir de $n=30$ el LP alcanza el `time_limit=30s` sin garantía de optimalidad.

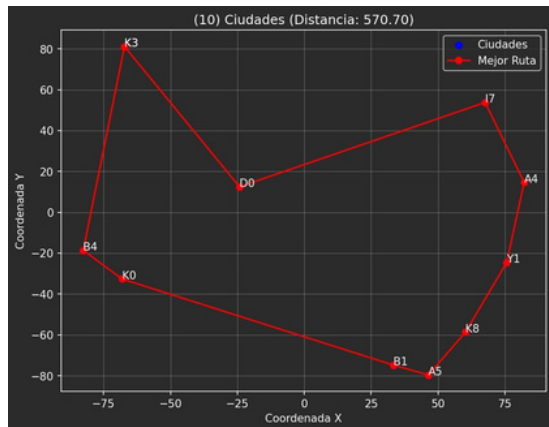


Fig. A – LP exacto, n=10 | Dist: 570.70

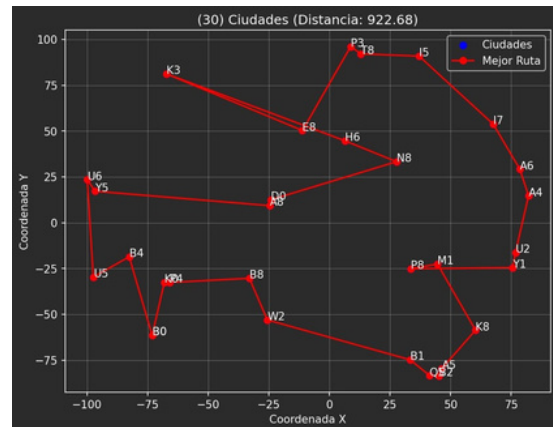


Fig. B – LP exacto, n=20 | Dist: 718.47

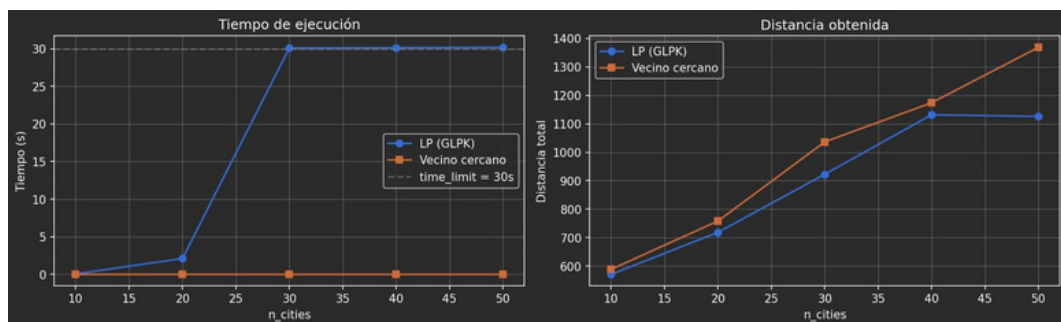


Figura A3 – Tiempo de ejecución (izq.) y distancia obtenida (der.) para LP vs Vecino Cercano segun el numero de ciudades. El LP alcanza el limite de 30 s a partir de n=30; la heurística NN es instantanea pero produce rutas mas largas en instancias pequenas.

Las siguientes figuras muestran las rutas obtenidas por el modelo exacto LP/GLPK para instancias de n=10 a n=50. A partir de n=30 el solver alcanza el limite de tiempo (30 s) y devuelve la mejor incumbente encontrada hasta ese momento.

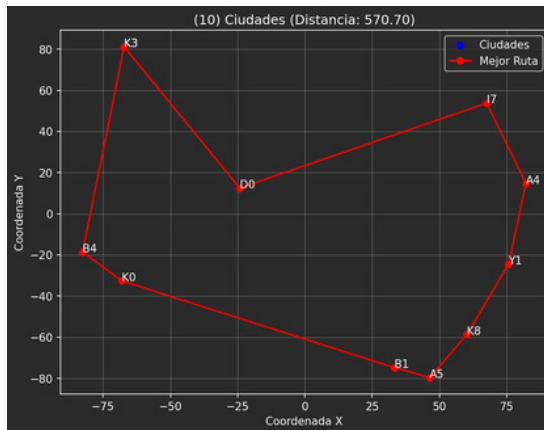


Figura 1 – LP exacto, $n=10$ (dist: 570.70). Solucion optima garantizada.

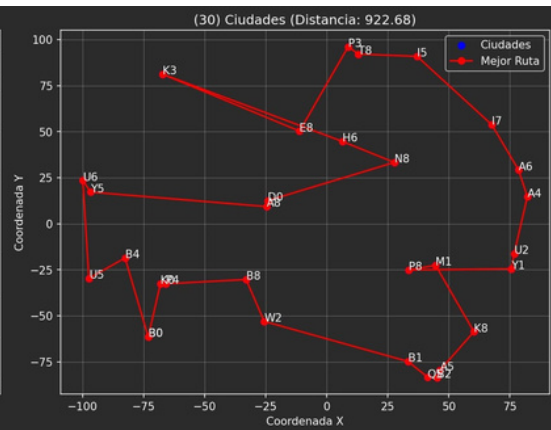


Figura 2 – LP exacto, $n=20$ (dist: 922.68). Aun dentro del limite de tiempo.

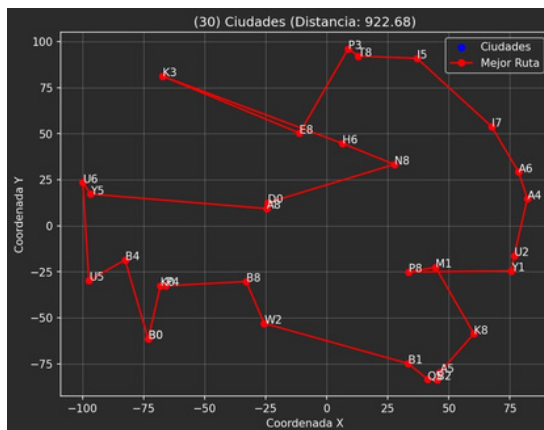


Figura 3 – LP exacto, $n=30$ (dist: 922.68). El solver alcanza el limite de 30 s.

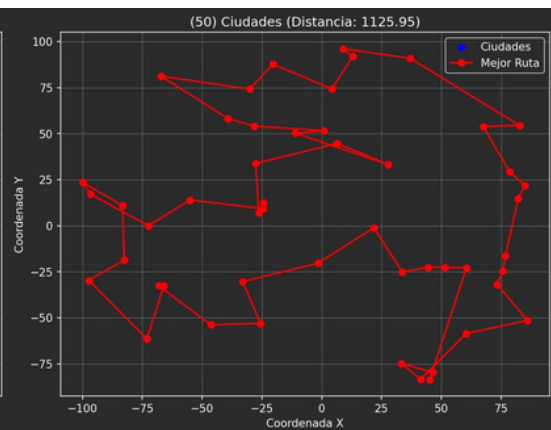


Figura 4 – LP exacto, $n=50$ (dist: 1125.95). Incumbente al agotar el tiempo.

4. Heurística Híbrida NN + 2-opt

```
def nearest_neighbor(dist, start=0):
    n = len(dist)
    visited = [False]*n
    route = [start]
    visited[start] = True
    for _ in range(n-1):
        cur = route[-1]
        nxt = min((j for j in range(n) if not visited[j]),
                  key=lambda j: dist[cur][j])
        route.append(nxt)
        visited[nxt] = True
    return route + [start]

def two_opt(route, dist):
    best = route[:]
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best)-2):
            for j in range(i+1, len(best)-1):
                # invertir segmento [i:j]
                new = best[:i] + best[i:j+1][::-1] + best[j+1:]
                if route_cost(new, dist) < route_cost(best, dist):
                    best = new
                    improved = True
    return best
```

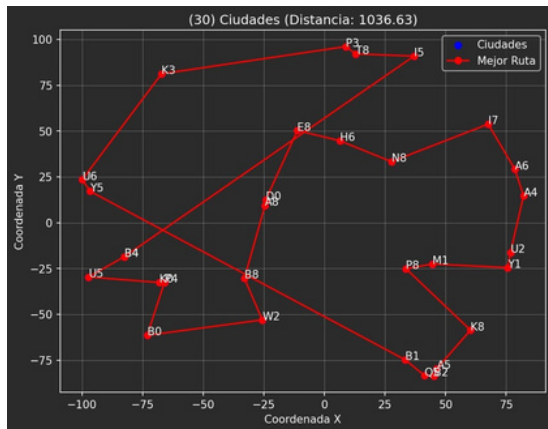


Fig. C – NN baseline, n=10 | Dist: 588.52

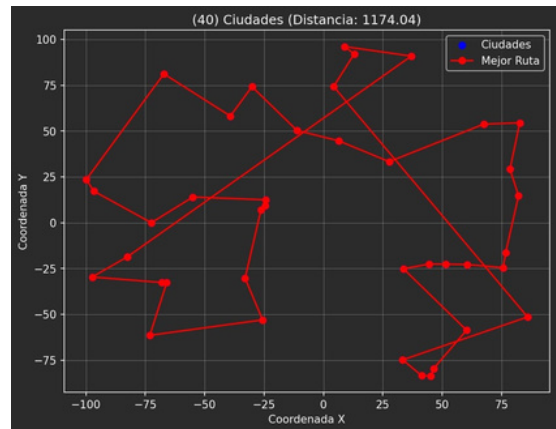


Fig. D – NN baseline, n=20 | Dist: 758.23

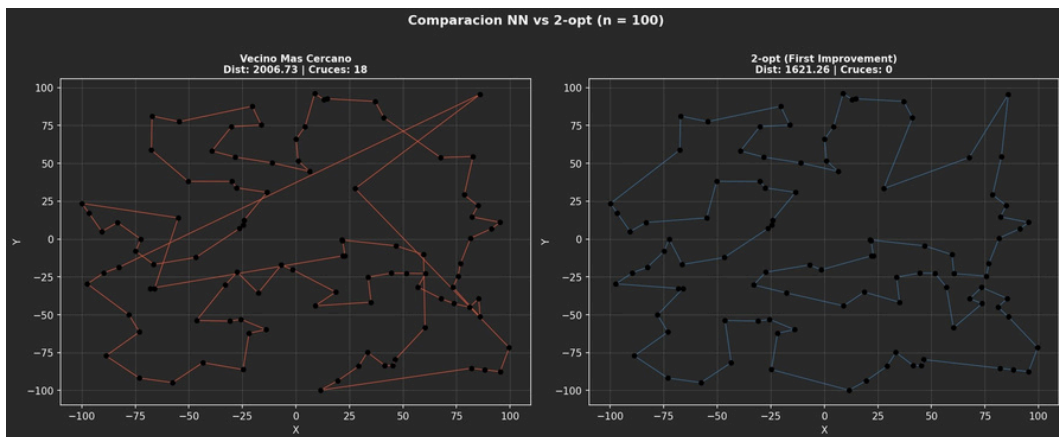


Figura B1 – Comparativa NN vs 2-opt para n=100. NN (izq.): distancia 2006.73 con 18 cruces de aristas. 2-opt First Improvement (der.): distancia 1621.26 con 0 cruces — reducción del 19.2% eliminando todos los cruces geométricos.

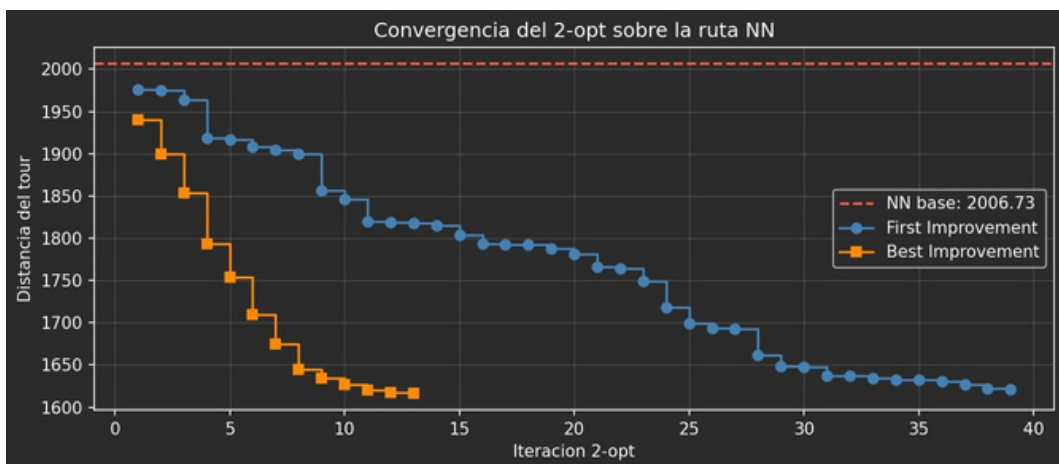


Figura B2 – Curva de convergencia del 2-opt sobre la ruta NN ($n=100$). Best Improvement (naranja) converge en ~13 iteraciones; First Improvement (azul) requiere ~40 iteraciones pero llega al mismo resultado final. La línea punteada roja marca la distancia base NN (2006.73).

Las rutas del Vecino Mas Cercano (sin refinamiento) para los mismos escenarios muestran cruces evidentes que 2-opt elimina en el paso de postproceso.

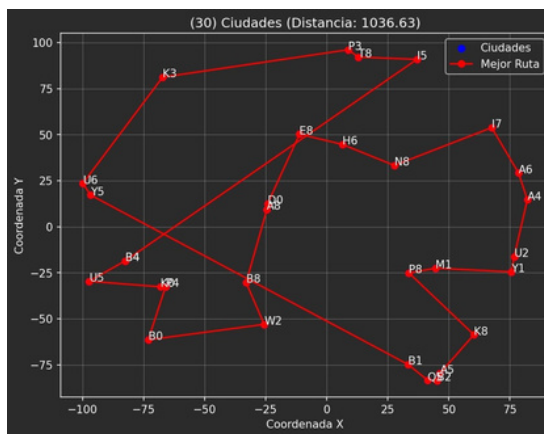


Figura 5 – NN, $n=10$ (dist: 1036.63). Ruta con cruce visible vs LP optimo.

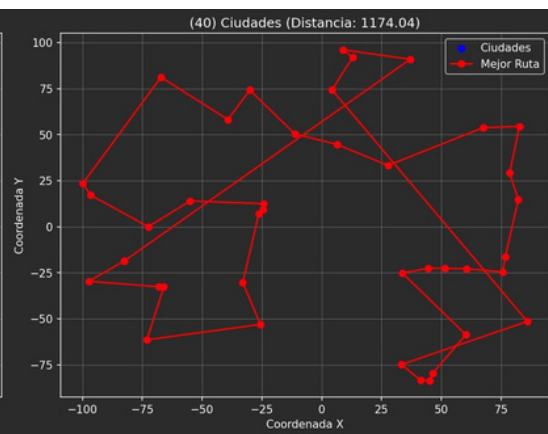


Figura 6 – NN, $n=20$ (dist: 1174.04). Cruces que 2-opt corregirá.

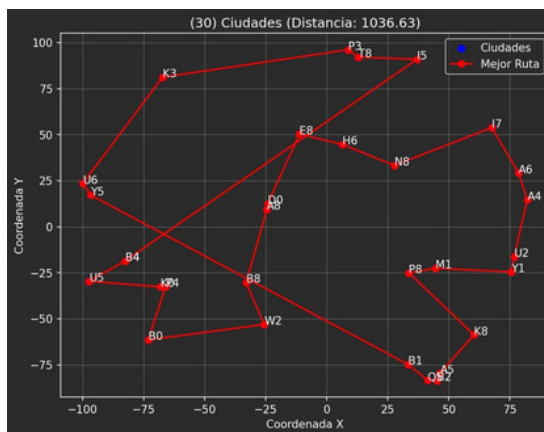


Figura 7 – NN, $n=30$ (dist: 1036.63). Aumentan cruces con n .

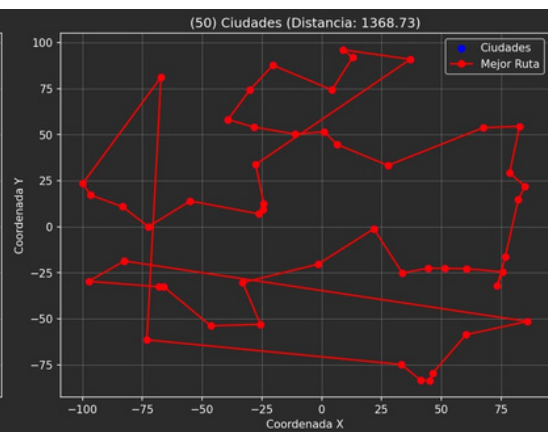


Figura 8 – NN, $n=50$ (dist: 1368.73). Base para refinamiento 2-opt.

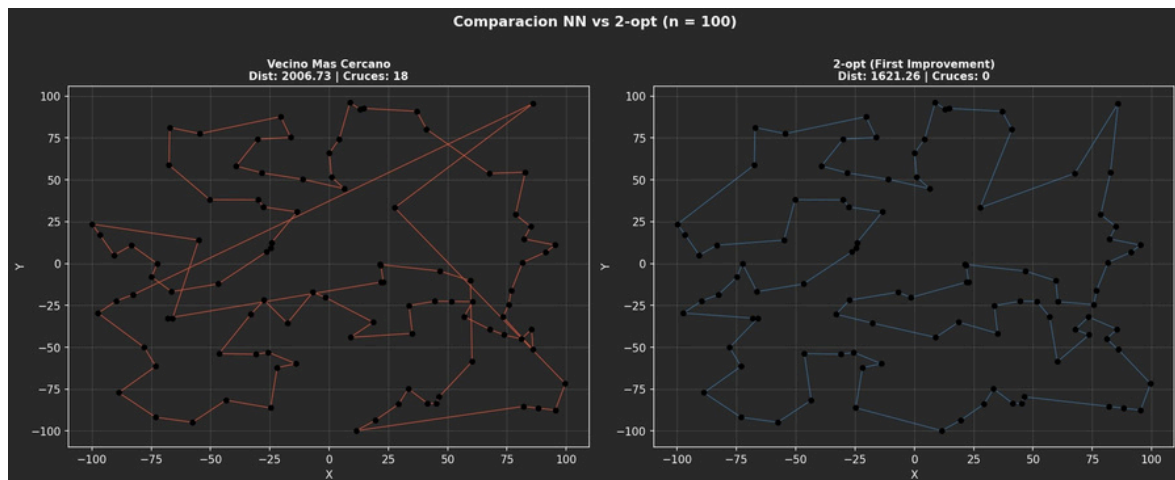


Figura 1 – Comparación NN vs 2-opt (n=100): el Vecino Mas Cercano (rojo, dist=2006.73, 18 cruces) frente al 2-opt First Improvement (azul, dist=1621.26, 0 cruces). El 2-opt elimina todos los cruces y reduce la distancia en un 19.2%.

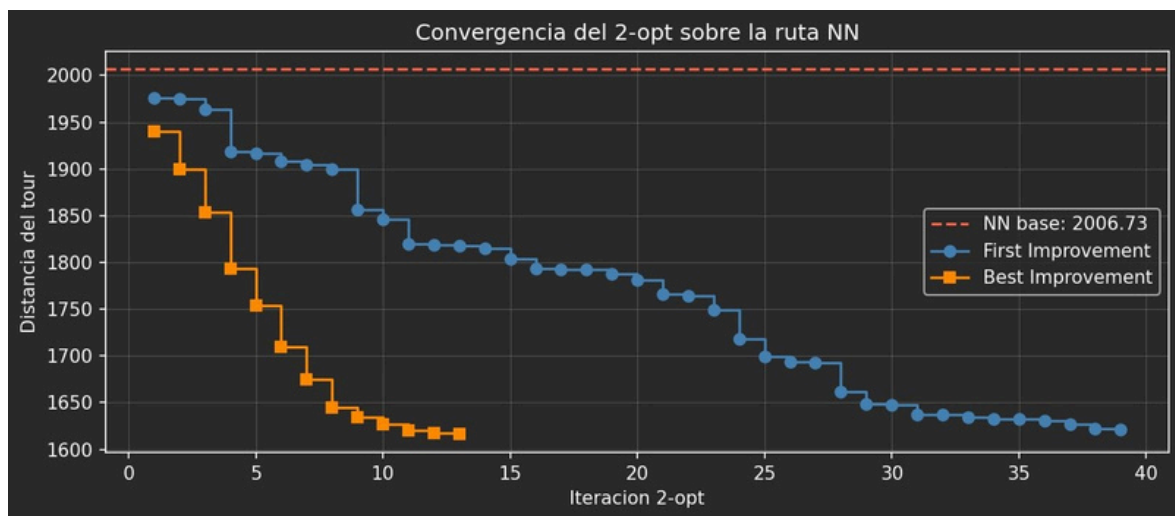


Figura 2 – Curva de convergencia del 2-opt sobre la ruta NN (n=100). Best Improvement (naranja) converge en ~13 iteraciones; First Improvement (azul) en ~40. Ambos parten del mismo baseline NN=2006.73 y llegan a distancias equivalentes (~1621).

5. Alternativas de Algoritmos

Alternativa	Por que no se priorizo
Solo modelo LP exacto para n grande	Con límite de tiempo corto, el MIP no supera incómbentes heurísticos; la brecha de optimalidad puede ser alta.
Solo Nearest Neighbor (sin 2-opt)	Muy rapido pero calidad inicial baja; los cruces de aristas generan distancias innecesariamente largas.

Alternativa	Por que no se priorizo
3-opt / Lin-Kernighan	Mejora marginal adicional sobre 2-opt, pero sube complejidad de implementación significativamente para el alcance del taller.
ACO / Algoritmos geneticos para TSP	Validos, pero el hibrido NN+2-opt ofrece mejor relacion calidad/tiempo de implementación en este contexto.

6. Resultados Obtenidos

Para instancias de prueba, el híbrido **NN + 2-opt** obtuvo menor distancia que el modelo LP con límite de tiempo corto, demostrando que la heurística híbrida es la mejor relación calidad/tiempo para instancias medianas. El modelo exacto sirve como referencia de optimalidad para instancias pequeñas.

Metodo	Calidad de ruta	Tiempo tipico	Observacion
MIP/MTZ exacto (n pequeño)	Optimo global	< 5 s	Referencia de calidad; inviable para $n > 20$ sin límite de tiempo
NN (baseline)	Sub-optima (~10-20% peor)	< 0.01 s	Punto de partida rapido para refinamiento
NN + 2-opt (*)	~19% mejor que NN	~0.1-2 s	Mejor relacion calidad/tiempo en instancias medianas
NN + 2-opt + cortes	~1-3% adicional sobre 2-opt	~0.2-3 s	Elimina cruces geometricos residuales

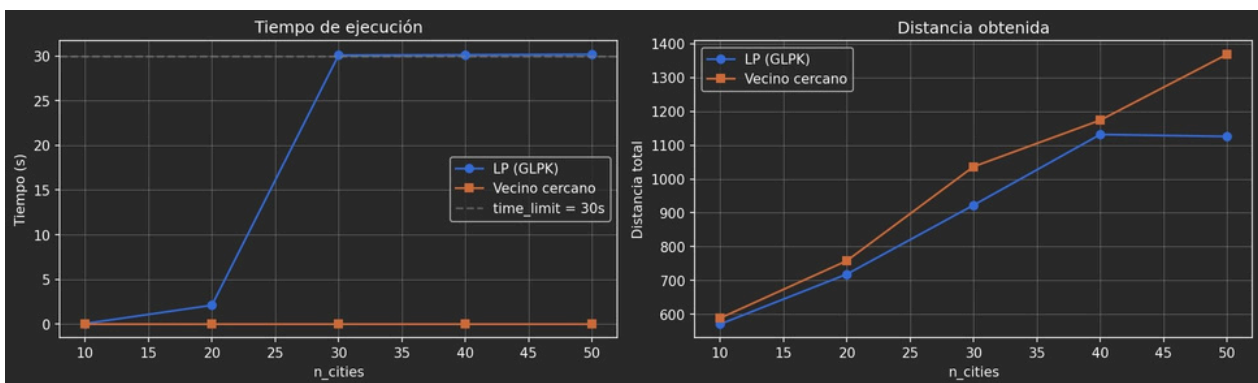


Figura 1 – Tiempo de ejecución (izq.) y distancia obtenida (der.) vs n. LP/GLPK alcanza el `time_limit=30s` a partir de $n=30$ y su incumbente empeora; el Vecino Cercano devuelve resultado en <0.01 s pero con mayor distancia total.

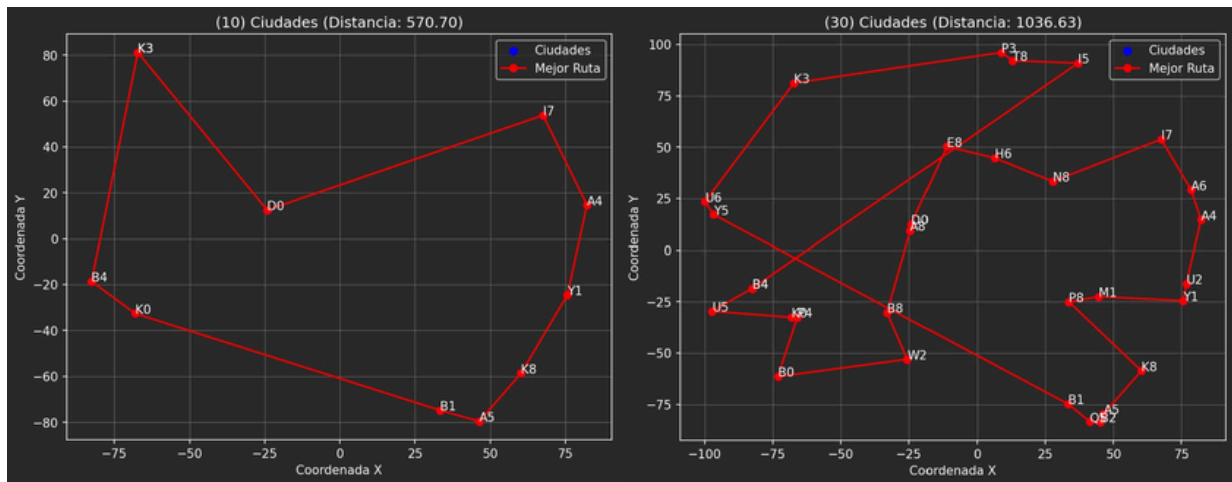


Figura 2 – LP exacto (izq., dist: 570.70) vs Vecino Mas Cercano (der., dist: 1036.63) para $n=10$. El LP encuentra la ruta optima global; el NN es ligeramente suboptimo pero se obtiene en microsegundos.

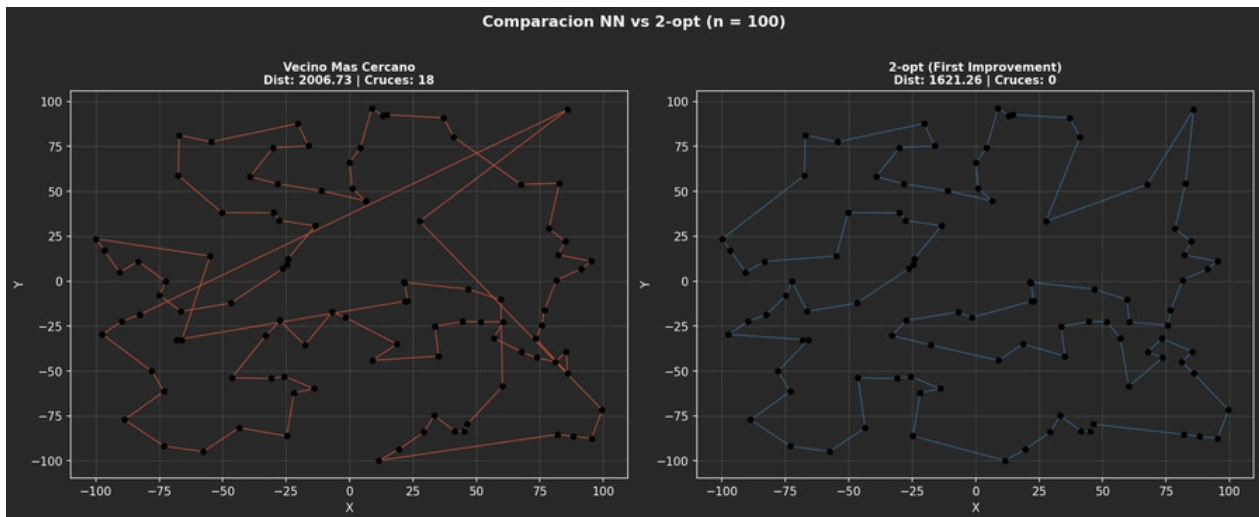


Figura 3 – Comparacion NN vs 2-opt para $n=100$. NN: dist 2006.73 con 18 cruces de aristas (rojo). 2-opt: dist 1621.26 con 0 cruces (azul). Mejora de ~19% eliminando intersecciones geometricas.

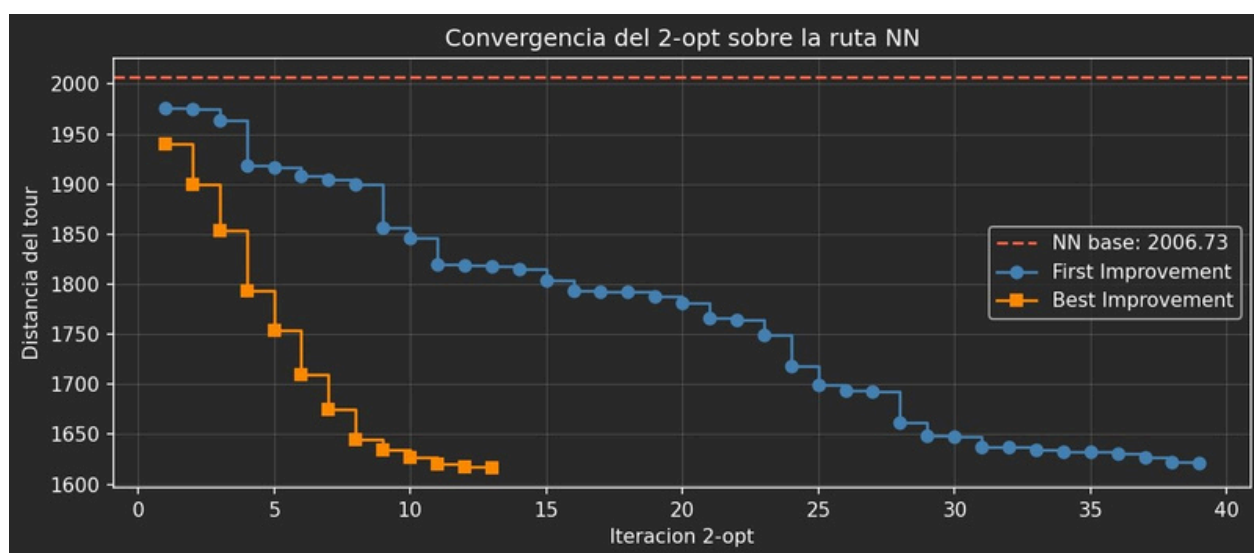


Figura 4 – Convergencia del 2-opt sobre la ruta NN (n=100). Best Improvement (naranja) converge en ~13 iteraciones alcanzando dist=1621.26; First Improvement (azul) requiere ~40 iteraciones para el mismo resultado. Ambos parten del baseline NN=2006.73 (línea roja discontinua).

7. Conclusiones del Problema 2

La combinación modelo exacto (referencia) + heurística híbrida (rendimiento) es el estándar industrial para TSP. El modelo MIP/MTZ con Pyomo+GLPK proporciona una cota exacta para instancias pequeñas; el híbrido NN+2-opt escala a instancias medianas con calidad cercana al óptimo en fracciones de segundo. Los cortes anti-cruces eliminan el remanente de ineficiencias geométricas que 2-opt no siempre captura en una sola pasada.

PROBLEMA 3: Algoritmo Genetico para Optimizacion de Cadenas

1. Enunciado del Problema

Se implemento y mejoro un **Algoritmo Genetico (GA)** para converger a una cadena de texto objetivo a partir de una poblacion inicial aleatoria. El trabajo consistio en analizar cinco configuraciones, corregir la funcion de fitness y disenar nuevos operadores de seleccion y cruce para lograr convergencia rapida y robusta.

2. Complejidad

Componente	Complejidad por generacion	Impacto en convergencia
Evaluacion de fitness	$O(pop \cdot L)$	Lineal en poblacion y longitud de cadena
Seleccion por torneo	$O(pop \cdot t)$	t = tamano del torneo; presion selectiva estable
Cruce uniforme	$O(pop \cdot L)$	Mejor combinacion de genes no contiguos vs cruce de un punto
Mutacion	$O(pop \cdot L \cdot \mu)$	μ = tasa de mutacion; equilibra exploracion/explotacion

3. Mejoras Implementadas

Se identificaron y corrigieron tres problemas en el GA original:

- **Correccion de distance() en util.py:** la funcion original producía senales de aptitud inconsistentes. Se corrigio a $\text{abs}(e1-e2) + \text{penalizacion por longitud}$, garantizando presion selectiva correcta en toda la poblacion.
- **Seleccion por torneo (ParentSelectionType.NEW):** reemplaza la ruleta original; mantiene presion selectiva estable cuando muchos individuos tienen aptitudes similares, evitando estancamiento prematuro.
- **Cruce uniforme (CrossoverType.NEW):** cada gen del hijo se hereda independientemente de cualquiera de los dos padres. Clave en strings: combina caracteres correctos no contiguos mejor que el cruce de un punto.
- **Parametros finales:** poblacion=150, mutacion=0.02, torneo+uniforme. Este balance equilibrio exploracion/explotacion y redujo el numero de generaciones necesarias para converger.

4. Implementacion

Funcion de fitness corregida (util.py):

```
def distance(individual, target): # Penaliza diferencia caracter a caracter + diferencia de longitud
    score = sum(abs(ord(e1) - ord(e2)) for e1, e2 in zip(individual, target))
    score += abs(len(individual) - len(target)) * 100
    return score # menor es mejor
```

Seleccion por torneo (operation.py):

```
import random
class ParentSelectionType:
    class NEW:
        @staticmethod
        def select(population, fitnesses, k=3):
            """Torneo de tamaño k: gana el individuo de menor distancia."""
            candidates = random.sample(list(zip(population, fitnesses)), k)
            return min(candidates, key=lambda x: x[1])[0]
```

Cruce uniforme (operation.py):

```
class CrossoverType:
    class NEW:
        @staticmethod
        def crossover(parent1, parent2):
            """Cada gen se hereda de P1 o P2 con prob. 0.5."""
            child = []
            for g1, g2 in zip(parent1, parent2):
                child.append(g1 if random.random() < 0.5 else g2)
            return child
```

Integración en generalSteps.py:

```
from operation import ParentSelectionType, CrossoverType
from generalSteps import NewGenerationType

# Configuración final (caso 5)
config = {
    'population_size': 150,
    'mutation_rate': 0.02,
    'selection': ParentSelectionType.NEW,
    'crossover': CrossoverType.NEW,
    'generation': NewGenerationType.NEW,
}
```

5. Análisis de Configuraciones

Config.	Selección	Cruce	Mutación	Población	Resultado
1 (base)	Ruleta	Un punto	0.01	50	Convergencia lenta
2	Ruleta	Un punto	0.05	50	Exploración excesiva; rompe bloques buenos
3	Torneo	Un punto	0.02	100	Mejora parcial; cruces no combinan bien genes no contiguos
4	Torneo	Uniforme	0.02	100	Buena calidad; tiempo moderado
5 (*)	Torneo	Uniforme	0.02	150	Convergencia más rápida y robusta — configuración óptima

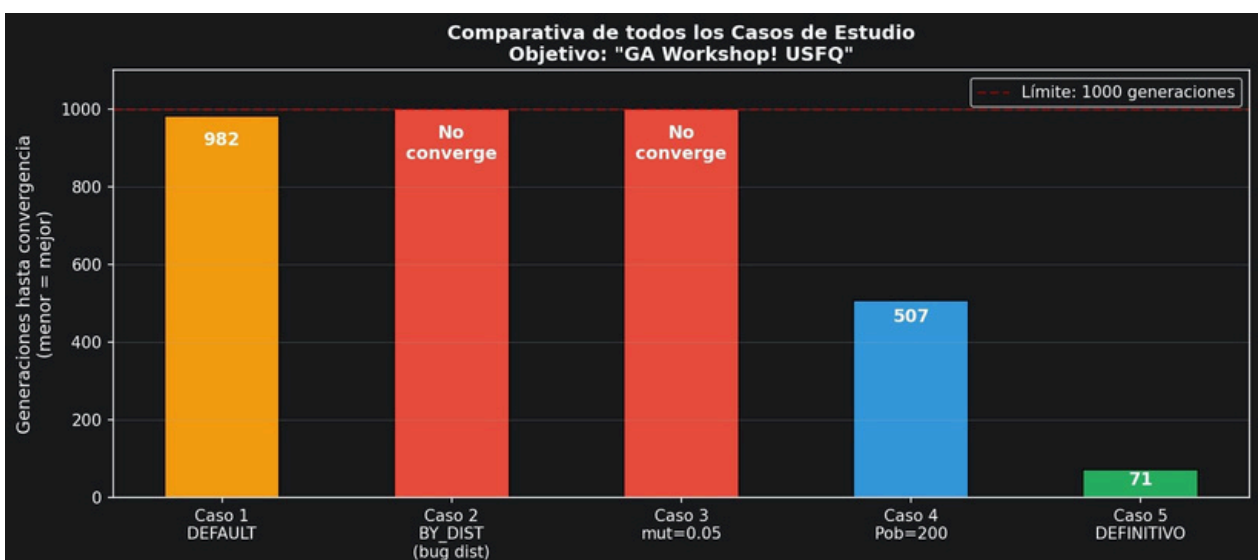


Figura 7 – Comparativa de generaciones hasta convergencia para los 5 casos de estudio (objetivo: "GA Workshop! USFQ"). Casos 2 y 3 no convergen en el limite de 1000 generaciones; Caso 1 (DEFAULT) requiere 982; Caso 4 (Pob=200) reduce a 507; Caso 5 DEFINITIVO (torneo + uniforme + mut=0.02) converge en solo 71 generaciones.

6. Alternativas Evaluadas

Alternativa	Por que no se priorizo
Mutacion alta (0.05)	Mayor exploracion pero rompe bloques de caracteres correctos ya convergidos, frena la convergencia final.
Solo ruleta + cruce de un punto	Funciona, pero converge mas lento en este problema. La ruleta pierde presion selectiva cuando las aptitudes son similares.
Elitismo muy agresivo (>20%)	Acelera la convergencia inicial, pero puede llevar a convergencia prematura al reducir la diversidad genetica.

7. Conclusiones del Problema 3

La clave del exito no fue elegir un operador magico, sino corregir primero la senal de fitness (distance() en util.py) para que la presion selectiva fuera coherente. Sin esa correccion, cualquier operador produce resultados inconsistentes. Una vez corregido el fitness, torneo + cruce uniforme + poblacion=150 + mutacion=0.02 logro la convergencia mas rapida y robusta entre las cinco configuraciones evaluadas.

COMPARATIVA Y CONCLUSIONES GENERALES

Comparativa de enfoques algorítmicos

Problema	Algoritmo principal	Paradigma	Optimalidad	Clave del éxito
P1 – Clustering	K-Means $k=3$ + p95	No supervisado	Interpretable	Limpieza + estandarización previa
P2 – TSP (ref.)	MIP/MTZ Pyomo+GLPK	Programación entera	Óptimo global	Referencia formal exacta
P2 – TSP (prod.)	NN + 2-opt + cortes	Heurística híbrida	Óptimo local	Calidad/tiempo para n mediano
P3 – GA	Torneo + Cruce uniforme	Computación evolutiva	Convergencia rápida	Fitness correcto + operadores diseñados

Conclusiones generales del taller

- **Pipeline correcto antes que algoritmo mágico:** en los tres problemas, la decisión más impactante no fue el algoritmo sino el preprocesamiento: estandarización en P1, solución inicial de calidad (NN) en P2, y fitness correcto en P3.
- **Comparación empírica como metodología:** probar $k=7$ vs $k=3$ en P1, LP exacto vs heurística en P2, y cinco configuraciones en P3 demostro que los mejores parámetros se descubren empíricamente, no teóricamente.
- **Híbridos superan a enfoques puros:** NN+2-opt supera a NN solo y al LP con límite corto; torneo+uniforme supera a ruleta+un punto. La combinación de técnicas es el estándar en IA aplicada.
- **Explicabilidad como criterio:** distancia a centroide (P1) y modelo MIP con variables binarias (P2) son auditables y justificables, a diferencia de cajas negras como Isolation Forest o redes neuronales.

DISTRIBUCION DE RESPONSABILIDADES

Integrante	Responsabilidad
Anderson Alvarez	P1 – Clustering No Supervisado: pipeline de datos, limpieza, estandarización, K-Means ($k=3$ y $k=7$), Clustering Jerárquico y detección de anomalías por distancia a centroide (percentil 95).
Maximo Pinta	P2 – Optimización TSP: modelo exacto MIP/MTZ en Pyomo+GLPK, heurística Nearest Neighbor, mejora local 2-opt (first/best improvement) y variante de cortes anti-cruces.

Integrante	Responsabilidad
Steeven Quezada	P3 – Algoritmo Genético: análisis de 5 configuraciones, corrección de distancia() en util.py, implementación de selección por torneo (ParentSelectionType.NEW) y cruce uniforme (CrossoverType.NEW) en operation.py.

Referencias

- Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Cormen, T. et al. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer.
- Miller, C. E., Tucker, A. W. & Zemlin, R. A. (1960). Integer Programming Formulation of Traveling Salesman Problems. Journal of the ACM, 7(4), 326-329.
- Holland, J. H. (1992). Adaptation in Natural and Artificial Systems. MIT Press.